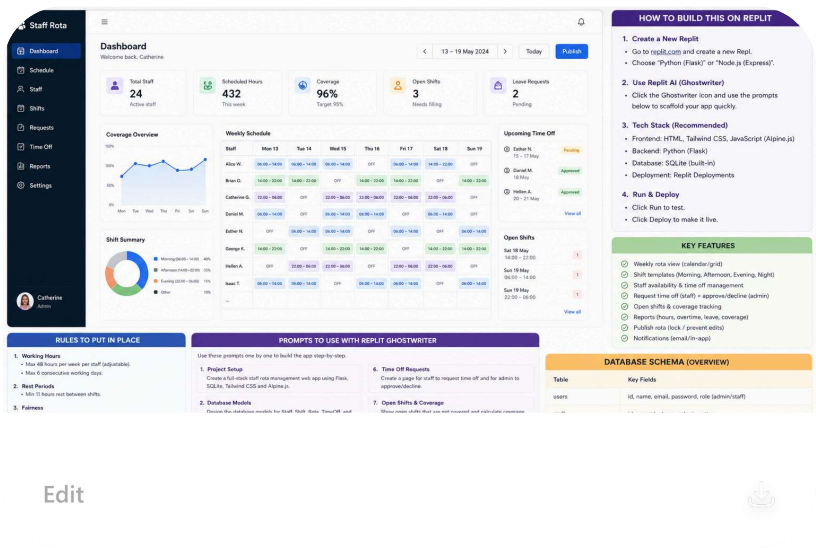


This is a copy of a shared ChatGPT conversation
Report conversation

Today 11:25 AM

📁 Uploaded a file

 **Create image** I need to create a dashboard on staff scheduling/rota to help me manage my staff planning. This will sit on Replit. Guide on how to go about this, what prompts or rules should I put in place?



I need to create a dashboard on staff scheduling/rota to help me manage my staff planning. This will sit on Replit. Guide on how to go about this, what prompts or rules should I put in place? same excel sheet but dont create an image

Based on your **Retail Operations Staff Rota.xlsx**, I would build this as a **staff planning application** rather than just a rota. Think of the rota as one module within a broader workforce planning dashboard.

A good Replit application should answer these questions immediately:

- Who is working today?
- Are we fully staffed?
- Who is on leave?

- Where are we understaffed?
- Who is approaching overtime?
- What does next week's schedule look like?

Recommended Dashboard Structure

1. Executive Dashboard

This is the first page.

Display KPI cards such as:

- Total Staff
- Scheduled Today
- Staff on Leave
- Open Shifts
- Weekly Hours Scheduled
- Coverage %
- Overtime Hours
- Labour Cost

Charts

- Staff scheduled per day
- Hours by department
- Leave trend
- Overtime trend
- Shift coverage

2. Weekly Rota

This becomes the main planning screen.

Example

Staff	Mon	Tue	Wed	Thu	Fri	Sat	Sun
Jane	Early	Early	Off	Late	Late	Early	Off
Peter	Late	Late	Late	Off	Early	Early	Off

Staff	Mon	Tue	Wed	Thu	Fri	Sat	Sun
Mary	Off	Early	Early	Early	Off	Late	Late

Features

- Drag and drop shifts
- Colour coded shifts
- Copy previous week
- Auto-fill recurring schedules
- Publish rota



3. Staff Directory

Each employee should have:

- Name
- Department
- Role
- Contract Hours
- Maximum Hours
- Availability
- Preferred Days
- Skills
- Phone
- Email
- Emergency Contact
- Employment Status
- Manager
- Store

4. Shift Library

Instead of typing times every week.

Example

Early
07:00-15:00

Middle
09:00-17:00

Late
14:00-22:00

Night
22:00-06:00

Off

Annual Leave

Sick Leave

Training

5. Leave Management

Staff request leave

Manager approves

Automatically blocks scheduling

Show

- Pending
- Approved
- Declined

6. Availability

Each employee can specify

Cannot work Mondays

Morning only

Weekend only

Maximum 4 days/week

School schedule

Part-time availability

Planner should respect this automatically.

7. Coverage Dashboard

Shows

Morning Coverage

Afternoon Coverage

Evening Coverage

Store Coverage

Department Coverage

Skill Coverage

Traffic light indicators



Fully staffed



Slightly understaffed



Critical

8. Reports

Useful reports include

Weekly hours

Monthly hours

Leave report

Overtime report

Labour cost

Attendance

Late arrivals

Coverage report

Suggested Database

Staff

StaffID

FirstName

LastName

Role

Department

ContractHours

MaxHours

Phone

Email

Status

Shifts

ShiftID

ShiftName

StartTime

EndTime

Colour

Hours

Rota

RotaID

StaffID

Date

ShiftID

Notes

Published

Leave

LeaveID
StaffID
StartDate
EndDate
Type
Status
Reason

Availability

AvailabilityID
StaffID
Day
Available
PreferredShift

Audit Log

User
Action
Date
Record Changed

Business Rules

These are the rules you should instruct Replit to enforce.

Working Hours

- Maximum 48 hours per week
- Maximum 6 consecutive working days
- Minimum 11 hours between shifts

- No double shifts

Fairness

Rotate

- weekends
- public holidays
- night shifts

Avoid assigning the same employee to all unpopular shifts.

Availability

Never schedule staff when marked unavailable.

Leave

Approved leave blocks scheduling automatically.

Skills

Only staff with required skills can work certain shifts.

Example

Cash Office

Supervisor

Manager

Forklift

First Aider

Coverage

Every shift should have

Minimum Staff

Ideal Staff

Maximum Staff

Display warnings when staffing falls below minimum.

Overtime

Warn when

- Contract hours exceeded
- Weekly hours exceeded
- Overtime threshold reached

Publishing

Once published

- Lock the rota
- Notify staff
- Require manager approval for changes

Audit Trail

Track

Who changed the rota

When it changed

What changed

AI Features to Build Later

Once the core system is stable, you can add AI capabilities such as:

- Suggest the best employee for an open shift.
- Recommend replacements for staff on leave.
- Balance hours fairly across the team.
- Detect likely understaffing before it occurs.
- Forecast staffing needs using historical data.
- Automatically generate the next week's rota while respecting all rules.

Master Prompt for Replit AI

Use this as your initial prompt in Replit AI:

Build a modern staff scheduling and workforce planning web application using Python (Flask), SQLite, HTML, Tailwind CSS, and JavaScript. The application should include an

executive dashboard, staff directory, weekly rota with drag-and-drop scheduling, shift templates, leave management, staff availability, coverage monitoring, overtime tracking, reporting, and audit logging. The system should support role-based access (Admin, Manager, Staff) and allow publishing and locking schedules. Business rules must enforce maximum weekly hours, minimum rest periods, staff availability, approved leave, skill requirements, fair shift rotation, and minimum staffing levels. Design the interface to be clean, responsive, and easy to use, with colour-coded shifts, KPI cards, charts, filters, search, and export to Excel/PDF. Structure the project with reusable components and prepare it for future AI-assisted scheduling.

Development Roadmap

To keep the project manageable, build it in phases:

1. **Phase 1:** Database schema, staff management, and authentication.
2. **Phase 2:** Weekly rota grid with shift templates.
3. **Phase 3:** Leave requests and availability management.
4. **Phase 4:** Dashboard KPIs, charts, and reports.
5. **Phase 5:** Business rule validation (hours, rest, coverage, fairness).
6. **Phase 6:** Notifications, publishing, audit logs, and AI-assisted scheduling.

This phased approach makes it much easier to build, test, and refine the application incrementally on Replit while keeping the codebase maintainable.